

Vocabulary as Moat

Microsoft DOS and the Politics of Proprietary Convention

Flyxion
Independent Researcher

2026

Abstract

This book traces MS-DOS's vocabulary, from command names to memory-management directives to file-recovery language, as a case study in how a proprietary interface can accumulate switching cost without any single coordinated decision to build one. Its earliest divergences from Unix (Chapter 1) are shown to be path-dependent inheritance from CP/M and DEC convention rather than a deliberate rejection of Unix precedent, using Microsoft's own published DOS 2.0 development sources to date the backslash decision precisely. Later chapters examine where that inherited vocabulary was sustained well past the point any compatibility rationale explained it: in DOS's command set (Chapter 2), its memory-management apparatus (Chapter 3), and its file-recovery tools (Chapter 4), each read against market evidence from competitors who demonstrated that simpler or more convergent alternatives were commercially available on the same hardware. A companion chapter (5) examines the inverse case, vocabulary DOS borrowed cleanly from Unix, to show where shared words concealed diverging mechanisms and where they did not. The book's synthesis (Chapter 6) proposes accumulated interface dependence, not deliberate lexical separation, as the best-supported mechanism, and states plainly what the evidence does not establish. A coda (Chapters 7 through 9) widens the frame to Microsoft's longer institutional history, from the 1976 Open Letter to Hobbyists through the Xenix divestment, the browser wars, and the 2018 re-release of MS-DOS's own source code, before closing with smaller observations on the graphical era, the retail logo culture, and the vocabulary of an unrelated language that DOS's naming happened to collide with.

Contents

Abstract	iii
1 From CP/M to PC-DOS	1
1.1 86-DOS and the CP/M Mandate	1
1.2 The Switch and the Slash	2
1.3 No Directories, No Conflict	2
1.4 The Argument This Sets Up	3
2 The Vocabulary of Enclosure	5
2.1 Full Words Against Terse Ones	5
2.2 DIR and the Suppression of a Precedent	6
2.3 Switches, Flags, and the Cost of Reconciliation	6
2.4 The Configuration Files as a Second Front	7
2.5 What This Chapter Does Not Claim	7
3 The Memory Management Wars	9
3.1 Expanded, Extended, and the Proliferation of Adjectives	9
3.2 HIMEM.SYS, EMM386, and the Grammar of DEVICE=	10
3.3 MemMaker and Automating Complexity, Not Removing It	11
3.4 What the Complexity Bought	11
4 Recovery, Verification, and the Vocabulary of Care	13
4.1 Undelete, Delete Sentry, and the Absence of a Unix Analogue	13
4.2 ScanDisk and the Compression of a Diagnostic Lineage	14
4.3 The Help System as a Second-Order Vocabulary	14
4.4 What a Late Arrival Demonstrates	15
5 Naming That Travels	17
5.1 Where the Words Actually Match	17
5.2 The 8.3 Boundary	18
5.3 Wildcards That Look Alike and Compute Differently	18
5.4 What Shared Vocabulary Is Evidence Of	19

I	Synthesis	21
6	Vocabulary as Moat	23
6.1	Restating the Evidence Without the Narrative	23
6.2	The Mechanism: Accumulated Switching Cost Without a Switching Decision	24
6.3	What the Moat Bought, and for Whom	24
6.4	The Limits of the Argument	25
II	Coda	27
7	The Longer Arc: From the Hobbyists to the Kernel	29
7.1	1976: The Letter Before the Company Had a Product to Enclose	29
7.2	1980–1987: A Real Unix, Held and Released	30
7.3	The 1990s: Embrace, Extend, and Extinguish	30
7.4	2014–2018: What Got Released, and When	31
7.5	The Lifecycle the Longer Arc Reveals	32
8	The Interior and the Seal	33
8.1	Windows as a Program DOS Ran	33
8.2	The Seal on the Box	34
8.3	The Weight of a Single Environment Variable	35
9	Zoom, Tutorial, Homophone	37
9.1	Paintbrush and the Pixel Beneath the Line	37
9.2	The Tutorial Taught the Hand a New Grammar	38
9.3	A Note on the Name	38
	Bibliography	41

Chapter 1

From CP/M to PC-DOS

The vocabulary that would distinguish Microsoft’s operating systems from the Unix tradition for the next four decades was not, in its origin, a Microsoft invention. Much of it was inherited, and much of the inheritance was contingent. Understanding that contingency matters because the later chapters argue that conventions can become an enclosure mechanism without having been invented for enclosure. The earliest divergences were made chiefly for compatibility with a smaller and more immediate system: Digital Research’s CP/M.

1.1 86-DOS and the CP/M Mandate

When Tim Paterson wrote QDOS, soon renamed 86-DOS, at Seattle Computer Products in 1980, his design brief was not to build an alternative to Unix. His constraint was narrower and more commercial: Seattle Computer Products needed an operating system for its Intel 8086 boards while Digital Research’s CP/M-86 remained unavailable. Paterson reproduced important CP/M programming conventions—especially the numbered system-call interface and File Control Block model—so that 8080 CP/M programs whose source was available could be ported to the 8086 with comparatively systematic translation. That was source-level portability, not binary compatibility and not a promise that every CP/M program could be converted mechanically without engineering work.¹

That decision explains much of DOS’s initial shape, but not every later vocabulary difference. CP/M supplied familiar commands, drive-letter addressing, 8.3 names, and a programming model that eased source ports. Microsoft first licensed 86-DOS and then purchased it in 1981 while supplying IBM’s version as PC-DOS. Compatibility and familiarity therefore exerted strong pressure, but they were not a complete design constitution: DOS 2.0 would soon import hierarchical directories, handles, pipes, and redirection from Unix-like practice.

It is worth pausing on how different this origin story is from Unix’s own. Unix’s command vocabulary, terse to the point of opacity (`cp`, `mv`, `rm`, `ls`), emerged at Bell Labs from a research culture that valued economy of keystrokes on slow teletypes and a philosophy, later

¹Tim Paterson’s retrospective account, “An Inside Look at MS-DOS,” describes the CP/M-facing design of 86-DOS; the surviving MS-DOS 1.25 and 2.0 sources and system-call documentation are published in Microsoft’s `microsoft/MS-DOS` repository.

articulated explicitly by Doug McIlroy and others, of small composable tools connected by pipes. Unix’s terseness was a design position, argued for and defended. CP/M’s conventions, and by inheritance DOS’s, were not argued for. They were simply what a small commercial team had settled on for a resource-constrained hobbyist machine, and what a second small commercial team later found it convenient not to change.

1.2 The Switch and the Slash

The genealogy of the forward slash requires more care than the familiar CP/M-to-DOS story suggests. CP/M’s own utilities were not governed by one uniform slash syntax: PIP, for example, placed options such as verification in brackets, as in `PIP B:=A:*. *[V]`. Slash-prefixed switches were familiar in DEC-influenced and other microcomputer software, and they became conventional in DOS commands, but the evidence does not justify deriving every DOS slash directly from CP/M. The safe historical claim is that DOS entered an ecosystem in which slash switches were already legible, then standardized them in its own command interface.

This detail matters because it complicates any simple story in which Microsoft chose the slash to differentiate itself from Unix. Unix’s dash-prefixed option (`ls -l`, `cp -r`) was itself a somewhat later standardization; early Unix commands were inconsistent about option syntax before the convention hardened over the 1970s. DOS’s slash and Unix’s dash are best understood not as competing proposals in a single debate but as parallel outcomes of two lineages, DEC-and-CP/M on one side, Bell Labs on the other, that had already diverged before either PC-DOS or a mature Unix option-parsing convention existed. What Microsoft did was not invent the slash. What Microsoft did, and what later chapters will treat as the real object of political interest, was decline every subsequent opportunity to reconsider it, even as the rationale for keeping it disappeared.

DOS did not call its slash-prefixed options “flags,” the term of art that had by then become standard in the Unix world and in the broader Bell Labs-descended tradition of documentation. It called them switches. The word was already present in CP/M documentation and in the DEC minicomputer manuals behind it, evoking a physical toggle on a piece of hardware rather than a bit set in a parsed argument vector. Whether this word choice carried any of the deliberate branding intent that this book will later argue characterizes later Microsoft naming decisions is doubtful for Chapter 1’s period; “switch” most likely traveled into DOS documentation simply because it was already the word CP/M and DEC manuals used, and nobody at Microsoft or IBM had reason in 1981 to replace inherited vocabulary that their target audience, largely CP/M refugees, already understood. The word’s persistence through every subsequent DOS revision, long after that audience had turned over, is a different matter, and one this book returns to in Chapter 3.

1.3 No Directories, No Conflict

The absence of any collision between the slash-as-switch convention and a slash-as-path-separator convention in early DOS was not a matter of foresight. It was a matter of DOS 1.0, like the CP/M it cloned, having no hierarchical filesystem at all. CP/M organized files into

flat, numbered “user areas” on a per-drive basis; there was no notion of nested directories, and therefore no need for any character to separate path components. DOS 1.0 for the IBM PC, released in 1981, inherited this flat structure along with everything else. A file was addressed by a drive letter and a filename, full stop. There was no path syntax to design, and consequently no reason for the slash’s role as a switch prefix to come under any pressure.

This changed in March 1983, with the release of PC-DOS 2.0. IBM’s PC XT, the hardware DOS 2.0 was built to support, shipped with a 10-megabyte hard disk, a capacity at which a flat file namespace becomes genuinely unworkable; users needed to organize hundreds of files into groups. Microsoft’s solution was to add a hierarchical directory tree, and the design of that tree owed an explicit debt to Unix. Microsoft’s own engineers, several of whom had worked on the company’s Xenix port of Unix for the 8086 in the same period, have described DOS 2.0’s directory structure as directly modeled on Unix’s, down to the use of `.` and `..` as entries denoting the current and parent directory. In this one respect, contrary to the entire pattern this book documents elsewhere, Microsoft looked at the Unix solution to a real problem and adopted it rather than inventing a parallel proprietary structure.

The surviving DOS 2.0 design material reveals that this resolution was not instantaneous. Microsoft’s own `SYSCALL.DOC` and `UTILITY.txt` examples use forward-slash paths such as `/sales/Mary/report` and `/F00/BAR`. DOS also contained a configurable switch character, evidence that the collision was recognized as an interface problem rather than a meta-physical property of the system. IBM-facing PC-DOS ultimately presented the backslash as the directory separator while retaining slash-prefixed switches, producing the familiar `C:\DOS\COMMAND.COM`. The backslash was therefore a compatibility settlement reached during DOS 2.0’s development, not simply a character Microsoft selected at the first moment directories were conceived.²

1.4 The Argument This Sets Up

The point of recounting this chain of inheritance in such detail is not antiquarian. It is to establish, before this book’s polemical argument begins in earnest, the distinction between path dependence and design intent. Nobody at Microsoft in 1981 sat down and decided that DOS should speak a different language than Unix in order to trap its users inside a proprietary vocabulary. The slash was CP/M’s before it was DOS’s, and CP/M’s before that it was DEC’s. The backslash was a patch, not a proposal. If the story ended in 1983, this would be a much shorter and much less interesting book: a footnote about legacy constraints in early microcomputer operating systems, of a kind that could be told about almost any software lineage.

The story does not end in 1983. What makes DOS a subject worth a monograph, rather than a paragraph in a history of CP/M, is what Microsoft did with this inherited vocabulary over the following decade, as the original compatibility rationale for each of these choices quietly expired while the choices themselves did not. CP/M software compatibility, the entire reason 86-DOS existed in its particular shape, was commercially irrelevant by the mid-1980s; DOS’s user base had long since become native to DOS rather than migrating from CP/M.

²Microsoft, *MS-DOS 2.0 System Calls Reference*, `v2.0/bin/SYSCALL.DOC`, and `v2.0/source/UTILITY.txt`, in the published MS-DOS source repository.

Unix, meanwhile, was no longer a research curiosity but an increasingly visible alternative, one that Microsoft's own Xenix division understood well enough to imitate selectively, as the directory tree shows. By the time of MS-DOS 5.0 and 6.0 in the early 1990s, the manuals excerpted throughout this book, there was no CP/M-compatibility reason left to call an option a switch, to prefix it with a slash, to spell `copy` rather than write `cp`, or to keep the drive letter as the unit of storage addressing rather than adopting something closer to a unified filesystem tree. The infrastructure of inheritance had become, by then, a matter of choice sustained rather than a constraint endured. Chapter 2 takes up that shift directly, examining how the path-separator problem, born of accident in 1983, became a template for a much more deliberate pattern of vocabulary divergence in the decade that followed.

Chapter 2

The Vocabulary of Enclosure

Chapter 1 established a distinction between path dependence and design intent. This chapter asks what happened after inherited conventions became familiar conventions in their own right. There is no evidence of a coordinated policy to prevent convergence with Unix, and difference alone cannot supply that evidence. The historically supportable claim is functional: repeated investment in DOS-specific names, documentation, and scripts made the dialect increasingly self-reinforcing, whether or not any individual naming decision was made for that purpose.

2.1 Full Words Against Terse Ones

The clearest single data point is the persistent divergence between DOS's spelled-out command names and Unix's abbreviated ones, and the divergence is worth examining command by command rather than as a generality, because the pattern is not simply "DOS uses long words." `COPY` where Unix has `cp`. `DEL` (and its synonym `ERASE`) where Unix has `rm`. `REN` (short for `RENAME`) where Unix has `mv` performing double duty as both move and rename. `DIR` where Unix has `ls`. `TYPE` where Unix has `cat`. `MD` and `RD` as abbreviations of `MKDIR` and `RMDIR`, which happen to coincide with Unix's own `mkdir` and `rmdir` almost exactly, spelled out in full where DOS itself offers a shorter alias.

That mixture is instructive, but not because it proves selective avoidance. Several DOS names came through CP/M: `DIR`, `ERA`, `REN`, and `TYPE` already belonged to the user vocabulary from which DOS drew. Others expressed operations differently from Unix: DOS exposed rename directly, while Unix's `mv` generalized movement and rename through one filesystem operation. `MD` and `RD` were aliases added alongside `MKDIR` and `RMDIR`; their resemblance to Unix therefore cuts against a theory of systematic lexical separation. The pattern is better described as compatibility first, followed by accretion, than as abbreviation everywhere except at points of possible overlap.

No surviving design document establishes that `COPY` was retained to avoid `cp`, and the names themselves cannot bear that inference. Inertia need not produce stylistic uniformity: it preserves different conventions inherited at different moments. What the command set does demonstrate is that a user learned an operational dialect rather than merely a collection of synonyms. Unix `mv` and DOS `REN` encoded different conceptual groupings, while shell

scripts, manuals, and habits accumulated around each grouping.

2.2 DIR and the Suppression of a Precedent

DOS's own history supplies a case that makes the selectivity easier to see. Digital Research, the same company whose CP/M supplied DOS's original interface, released DR-DOS in 1988 as a direct MS-DOS competitor, largely command-line compatible with Microsoft's product but distinguished in places by its own choices. DR-DOS did not restructure its core commands around Unix vocabulary either; Digital Research had every commercial reason to remain compatible with the MS-DOS conventions its customers already knew, and largely did. The interesting comparison is not DR-DOS's core command set, which tracked MS-DOS closely for compatibility reasons of its own, but the fact that a genuine alternative existed at all and chose, when it did differ, to differ in directions unrelated to Unix convergence (multitasking extensions, a task-switching shell, memory management utilities) rather than toward it.

DR-DOS actually demonstrates how strong compatibility pressure had become. A commercially viable DOS competitor needed existing batch files, applications, and users' commands to continue working, so retaining **DIR** was not interchangeable with adopting **ls**. Digital Research could innovate in memory management or multitasking while preserving the command surface on which compatibility claims depended. Microsoft did not have to prohibit alternative vocabulary; the installed base disciplined competitors on its behalf. That is a genuine moat effect, but it arose from interface compatibility rather than evidence of suppressed Unix precedents.

2.3 Switches, Flags, and the Cost of Reconciliation

Chapter 1 traced the word “switch” to CP/M and DEC documentation predating any Microsoft involvement, and noted that its persistence past the expiration of the compatibility rationale was a separate question from its origin. That question can now be answered more directly. By MS-DOS 3.0 (1984) and especially MS-DOS 5.0 (1991), the manuals this book draws on throughout, “switch” was not merely an inherited word coasting on inertia. It had become the term structuring an entire documentation apparatus: command reference pages organized around a **Switches** heading, error messages referring users to “valid switches,” and a help system (**HELP** in DOS 5.0, expanding on the earlier **DOS Shell** documentation) that taught the word to a generation of users who had never touched CP/M or a DEC minicomputer and had no continuity-of-vocabulary reason to inherit it.

Changing a manual from “switch” to “flag” would not have broken the parser, but it would have imposed a documentation and retraining cost while offering DOS users little immediate benefit. Unix itself used *option* as well as *flag*, so there was no single external term with which DOS was obliged to reconcile. The persistence of “switch” is therefore best read as institutional vocabulary: once repeated across manuals, help screens, books, classrooms, and support calls, a word becomes infrastructure even when software does not parse it.

2.4 The Configuration Files as a Second Front

The clearest instance of a DOS-specific configuration dialect is the pair `CONFIG.SYS` and `AUTOEXEC.BAT`. `AUTOEXEC.BAT` existed in the DOS 1.x line; `CONFIG.SYS` and installable device-driver configuration arrived with DOS 2.0. `CONFIG.SYS` used directives like `DEVICE=`, `FILES=`, and `BUFFERS=` to configure driver loading and system resources at boot. `AUTOEXEC.BAT` used the DOS batch language, including `IF`, `GOTO`, `FOR`, and percent-delimited variables. These mechanisms differed from Unix kernel configuration and shell startup files, but solving analogous tasks does not create a presumption that their syntax ought to converge.

This pairing is worth dwelling on because it was new territory as of 1983, not inherited from CP/M, which had neither file. Every choice made in designing `CONFIG.SYS`'s directive syntax and the batch language's control structures was made after Microsoft's engineers had already built the Unix-derived directory tree described in Chapter 1, and after several of those same engineers had worked on Xenix, Microsoft's own Unix port, meaning direct familiarity with Unix's `/etc` conventions and shell scripting syntax was present on the team at the moment these decisions were made. Unlike the slash, there was no already-shipped software forcing `CONFIG.SYS`'s directive syntax or the batch language's percent-sign variable convention into a specific shape; these were greenfield designs, built from scratch to solve problems (driver loading order, startup automation) that Unix had already solved with its own vocabulary, available to Microsoft's engineers as a known reference. The dollar-sign convention for shell variables, established Unix practice well before 1983, was not adopted. The percent sign was chosen instead, for reasons no surviving documentation explains, in a decision with no CP/M precedent to point to and no compatibility constraint forcing the outcome.

2.5 What This Chapter Does Not Claim

It is worth being precise about the limits of the argument built across this chapter, because the temptation toward a stronger and less defensible claim is real. This book does not claim that Microsoft's engineers sat in a room in 1983 and voted to reject Unix vocabulary as a matter of explicit competitive strategy against a system, Unix, that was not yet Microsoft's meaningful competitor in the personal computer market at that time. The evidence assembled here is compatible with a more mundane mechanism: an engineering culture that, once it had already diverged from Unix for defensible reasons in 1980 to 1983, found it individually costless in each subsequent instance to continue diverging, because doing so required no active decision, only the absence of one. Sustaining a divergent vocabulary demanded nothing from anyone. Reconciling it would have demanded, at minimum, that someone notice the divergence, judge it worth fixing, and accept whatever documentation and retraining cost fixing it entailed, for a benefit (interoperability with a system DOS's own users mostly did not use) that was diffuse and largely external to Microsoft's own commercial position.

What the chapter does claim is narrower and, this book will argue, more durable: that the absence of reconciliation across a full decade, at points (documentation terminology, greenfield configuration-file design) where reconciliation carried no technical cost, cannot be explained by the path-dependence account that fully explains Chapter 1's 1983 backslash

decision. Something had changed between 1983 and 1991. What had changed was not that vocabulary divergence stopped being an accident and started being a plan, in the sense of a document Microsoft executed. What had changed was that the divergence had become an asset, whether anyone at Microsoft named it as one or not, in a specific sense the next chapter develops directly: a user, an IT department, or an enterprise fully fluent in DOS's `switch`, `CONFIG.SYS`, and `AUTOEXEC.BAT` vocabulary had accumulated knowledge with no transfer value to any Unix system, and every year that vocabulary went unreconciled increased the cost of ever learning the alternative. Chapter 3 turns to `CONFIG.SYS` and `AUTOEXEC.BAT` in more technical depth, and to the years surrounding MS-DOS 6.0's memory-management wars, to argue that configuration complexity itself, not merely naming, became a proprietary ecosystem Microsoft had strong commercial reasons not to simplify toward any external standard.

Chapter 3

The Memory Management Wars

Chapter 2 closed by proposing that configuration complexity itself, not merely the vocabulary used to express it, became a proprietary asset in the DOS ecosystem. This chapter tests that proposal against the single episode in DOS's history where configuration complexity was most visible, most consequential, and most thoroughly documented: the struggle throughout the second half of the 1980s and into the early 1990s to work around the 640-kilobyte conventional memory barrier, an episode DOS users of the period universally remember and DOS manuals of the period devote entire chapters to. It is worth stating at the outset what this chapter does and does not claim about that barrier. The 640-kilobyte limit itself was not a DOS design choice at all; it was a consequence of the IBM PC's 1981 memory map, which reserved the address space above 640K for video buffers, ROM, and adapter cards, a hardware decision DOS had to live inside rather than one it imposed. What this chapter is interested in is not the barrier's origin but the vocabulary and configuration apparatus DOS built to manage life around it, apparatus that, like `CONFIG.SYS` and `AUTOEXEC.BAT` themselves, was greenfield work built well after Unix's own approach to memory management was available as a reference, and that took a shape with no Unix analogue at all.

3.1 Expanded, Extended, and the Proliferation of Adjectives

By the mid-1980s two incompatible schemes existed for giving DOS programs access to memory beyond the 640-kilobyte barrier, and the vocabulary DOS accumulated to distinguish them is itself a small case study in the pattern this book has been tracing. *Expanded memory*, standardized as the Lotus-Intel-Microsoft Expanded Memory Specification (EMS) in 1985, worked by bank-switching blocks of additional physical memory into a small window inside the conventional 640K space, a technique with real hardware-driver complexity (an EMS board, or later a software emulator, had to intercept and remap addresses on the fly) that had no counterpart in the memory architecture of the Unix minicomputers and workstations of the period, which addressed memory directly rather than through a switched window. *Extended memory*, by contrast, was simply the physical address space above one megabyte that the 80286 and later processors could address directly in protected mode, closer in spirit to ordinary linear addressing, but inaccessible to DOS programs running in the 8086 real mode DOS itself used, which meant it required its own access layer, standardized later and

more loosely as the Extended Memory Specification (XMS) in 1988.

That DOS ended up with two differently-named, differently-behaved, industry-consortium-specified memory types, EMS and XMS, each requiring its own driver, its own `CONFIG.SYS` directive, and its own application-level support before a program could use it, was in the first instance a hardware and standards-body fact rather than a Microsoft vocabulary choice; Microsoft was one signatory among several on the EMS specification, not its sole author. What belongs to this book's argument is what came next: the layer of driver names, directive syntax, and diagnostic terminology DOS itself built on top of this already-fragmented hardware landscape, a layer that was entirely within Microsoft's control and that, like the configuration files examined in Chapter 2, took a shape unrelated to any external convention.

3.2 HIMEM.SYS, EMM386, and the Grammar of `DEVICE=`

The central artifact of this chapter's argument is a single line of `CONFIG.SYS` syntax, repeated with variations across millions of PCs by the early 1990s: `DEVICE=C:\DOS\HIMEM.SYS`, typically followed by `DEVICE=C:\DOS\EMM386.EXE NOEMS` and `DOS=HIGH,UMB`. Each piece of this grammar deserves attention. `HIMEM.SYS`, introduced with DOS 4.0 in 1988 and refined through DOS 5.0 in 1991, was Microsoft's XMS driver, the software that made extended memory usable at all; its name, a compressed portmanteau of "high memory," followed the same telegraphic driver-naming convention DOS had used since its earliest device drivers (`ANSI.SYS`, `VDISK.SYS`) rather than adopting anything resembling a Unix kernel module's naming or loading syntax. `EMM386.EXE`, DOS 5.0's expanded-memory manager, performed the more remarkable trick of emulating the EMS window using the 80386's virtual-86 mode, letting a machine with only extended memory installed still run software written for expanded memory, a genuinely clever piece of systems engineering with, again, no parallel need or parallel vocabulary in the Unix world, where no equivalent hardware-window emulation problem existed to be solved.

The directive `DOS=HIGH,UMB` is the piece most worth dwelling on, because it names two distinct and non-obvious concepts with two compressed, DOS-specific abbreviations that a decade of manuals never expanded into anything more transparent. `HIGH` instructed DOS to load its own kernel into the High Memory Area, a specific 64-kilobyte region just above the one-megabyte boundary reachable from real mode by an addressing quirk of the 8086 (the so-called A20 line), freeing conventional memory that DOS's own kernel would otherwise have occupied. `UMB` instructed DOS to make Upper Memory Blocks, the unused portions of the 640K-to-1MB reserved region that `EMM386.EXE` could remap into usable space, available for loading device drivers and terminate-and-stay-resident programs. Both concepts, the High Memory Area and Upper Memory Blocks, were genuine DOS-specific engineering achievements, exploiting quirks of the specific 8086-family hardware DOS ran on; this book does not begrudge Microsoft the invention of a vocabulary to describe them, since no pre-existing Unix term could have named them, Unix systems of the period having no equivalent hardware constraint to work around in the first place. What is worth noting is the density of unglossed acronym this single directive line asks a user to hold: `DOS`, `HIGH`, `UMB` function together, in the space of eleven characters, as a small proprietary language whose referents (kernel relocation, memory remapping, a hardware addressing quirk) are three conceptually separate operations compressed into a syntax that gives a new user no

purchase on any of them without consulting the manual.

3.3 MemMaker and Automating Complexity, Not Removing It

DOS 6.0, released in 1993, introduced **MEMMAKER**, a utility that automated the tuning of **CONFIG.SYS** and **AUTOEXEC.BAT** for conventional-memory availability. **MEMMAKER** is revealing because it automated an established interface rather than replacing it: the utility tested configurations across reboots and then wrote the same **DEVICEHIGH**, **LOADHIGH**, and related directives an experienced user might have written by hand. That does not prove Microsoft could have abstracted away every hardware conflict; drivers and adapter memory maps were genuinely variable. It does show how compatibility favors automation around a mature grammar. Replacing that grammar would risk old drivers and documentation, while generating it preserved both.

This matters for the chapter's argument because a genuine alternative existed and was commercially demonstrated in the same market. Quarterdeck Office Systems's **QEMM**, a third-party memory manager first released in 1987 and dominant enough by the early 1990s that DOS 6.0's own **MEMMAKER** was widely understood, including in the trade press of the period, as Microsoft's belated response to it, offered more aggressive and more automatic memory optimization than Microsoft's own tools for several years running, and did so as an add-on product sold directly to users frustrated with DOS's native configuration burden. Digital Research's **DR-DOS**, the same competitor discussed in Chapter 2, shipped memory management, including its own upper-memory-block loading, as a more integrated and by most contemporary accounts less manually intensive feature well before MS-DOS 6.0 caught up. That two competitors, one a pure utility vendor with no operating system of its own to sell, one a full DOS-compatible competitor, both found commercial success by making DOS memory management simpler than Microsoft's own tools made it, is direct market evidence that the complexity **CONFIG.SYS** and **AUTOEXEC.BAT** asked users to manage was not a necessary consequence of the underlying 8086 memory architecture. It was a consequence of how Microsoft chose to expose that architecture to users, and other vendors showed a less demanding exposure was buildable on the identical hardware.

3.4 What the Complexity Bought

The chapter's title borrows the martial framing the trade press of the period used, unreservedly, to describe this episode, and the framing was apt in one respect this book wants to preserve: real competitive stakes attached to who controlled memory management on the DOS platform. A user fluent in **HIMEM.SYS**, **EMM386**, and the specific ordering discipline that made a working **CONFIG.SYS** on a given machine's specific mix of drivers, and the IT departments and consultants who built businesses around that fluency, possessed knowledge with almost no transfer value outside the DOS platform and comparatively little transfer value even across a DOS machine's hardware variations, since the correct upper-memory-block assignment for one machine's specific card configuration frequently did not work on another. This is the same mechanism Chapter 2 identified in slower motion at the level of

individual command names, operating here at the level of an entire configuration subsystem: complexity that was genuinely rooted in real hardware constraints, genuinely difficult to eliminate entirely, and yet demonstrably more difficult in Microsoft's own tools than in a directly competing product built on the same hardware.

Whether this asymmetry was a deliberate strategic choice, a genuine engineering lag behind a more focused competitor, or, as this book's overall argument suggests is likeliest, some blend in which an already-established DOS-specific vocabulary and configuration model made the DOS-consistent solution (extend `CONFIG.SYS`'s existing grammar) the path of least internal resistance regardless of whether it was the path of least user burden, is not a question this chapter can settle from the documentary record alone. What the chapter has established is narrower and sufficient for the book's cumulative case: that a genuinely simpler configuration model for the same hardware problem was commercially available from competitors throughout the period Microsoft's own tools remained more demanding, which rules out hardware necessity as a complete explanation and leaves institutional inertia, and the accumulated value of DOS-specific fluency to Microsoft's own platform position, as the remaining candidates. Part IV takes up the question of how to weigh those candidates against each other directly, drawing together the vocabulary evidence of Chapter 2 and the configuration evidence of this chapter into the book's synthesis.

Chapter 4

Recovery, Verification, and the Vocabulary of Care

Chapters 2 and 3 traced this book’s pattern, vocabulary and configuration complexity persisting past the point where any compatibility rationale explained them, through DOS’s command set and its memory-management apparatus. This chapter locates the same pattern in a third domain, one that arrived later in DOS’s lifecycle and therefore offers the clearest evidence yet that the pattern was not simply an artifact of DOS’s 1981 origins: file recovery and disk verification, as DOS 5.0 through 6.22 built them out between 1991 and 1994. The source base for this chapter is the *Microsoft MS-DOS Concise User’s Guide*, a manual distributed with mid-1990s consumer MS-DOS releases, whose own language supplies the clearest evidence of how thoroughly DOS’s documentation had by this point developed a self-contained idiom with no Unix referent at all.

4.1 Undelete, Delete Sentry, and the Absence of a Unix Analogue

Unix’s `rm` command has, since the earliest research versions of the system, deleted a file by removing its directory entry, an operation with no undo and, absent a separate backup regime, no recovery path; the terseness of the command name is matched by the terseness of its consequence. DOS took a different path, and did so relatively late: `UNDELETE` first appeared with DOS 5.0 in 1991, ten years into the system’s life, as a utility that could recover a deleted file by scanning the file allocation table for entries DOS had marked deleted but not yet overwritten, a recovery method inherently probabilistic and dependent on how much disk activity had occurred since the deletion.

DOS 6.0, in 1993, extended this with a feature the *Concise User’s Guide* calls Delete Sentry, a background utility that, once installed, intercepted deletions and moved the deleted file’s contents into a hidden cache directory rather than simply marking its space free, guaranteeing recoverability (subject to available disk space) rather than merely permitting it opportunistically. The manual’s own description of the resulting state is worth citing directly, because the specific word it reaches for is not a term of art borrowed from any prior operating system tradition but a small piece of invented product vocabulary: files protected

by Delete Sentry, the guide instructs, appear in the Undelete screen “in Perfect condition,”¹ a capitalized quality rating with no counterpart in DOS’s own file-attribute vocabulary (which elsewhere used plain, lower-case, technical terms: read-only, hidden, archive) and certainly none in the Unix tradition, where a file simply exists or does not.

“Perfect condition” repays attention because of what kind of word it is. It is not a technical description of the recovery mechanism, since a Unix administrator restoring a file from an equally complete backup would not think to describe the result in evaluative language at all; the file is simply restored, correct, identical to what was deleted. Microsoft’s documentation, faced with the identical technical fact, a byte-for-byte intact deleted file, reached instead for a consumer-product register: a rating scale implying gradations (one can infer, without the manual stating it directly, that files recovered through the FAT-scanning method rather than Delete Sentry’s cache would occupy some lesser condition on an implicit scale) borrowed from the vocabulary of physical goods and their resale value rather than from any operating-system tradition, Unix’s or DOS’s own prior thirteen years of documentation included.

4.2 ScanDisk and the Compression of a Diagnostic Lineage

Unix’s disk-checking utility, `fsck` (file system consistency check), dates to the earliest Unix filesystem implementations and has retained its terse, function-describing name across four decades and a wide range of underlying filesystem formats; the name states plainly what the tool does and nothing more. DOS’s equivalent utility went through two names in quick succession that illustrate the same selective-abbreviation pattern Chapter 2 documented at the command level. `CHKDSK` (check disk), present since early DOS versions, followed the same MD/RD-style abbreviation convention, compressed but not spelled out in full, and, notably, not colliding with `fsck` despite `CHKDSK`’s own six-letter compression leaving ample room for a coincidental match that never happened. DOS 6.2, in late 1993, replaced `CHKDSK` as the recommended tool with `SCANDISK`, a longer, fully-spelled, product-style name rather than a further compression, reversing the abbreviating direction the DOS lineage had otherwise favored and moving toward the same evaluative, consumer-facing register as “Perfect condition.” The *Concise User’s Guide*’s own framing supports this reading: it describes ScanDisk as a utility that can “check your hard disk for errors, and repair it if errors are detected,”² language pitched at a consumer rather than a systems-programming audience, in contrast to `fsck`’s terser and more technical presentation in Unix manuals of the same period, which typically assumed the reader already understood what filesystem consistency meant and needed only the command’s flags explained.

4.3 The Help System as a Second-Order Vocabulary

Both the Delete Sentry and ScanDisk passages point to a further layer of DOS-specific vocabulary this book has not yet examined directly: the built-in help system itself, invoked

¹*Microsoft MS-DOS Concise User’s Guide*, p. 39, “Files protected by Delete Sentry are in Perfect condition.”

²*Microsoft MS-DOS Concise User’s Guide*, p. 39.

in DOS 5.0 and later as `help` followed by a command name, a convention the *Concise User's Guide* uses repeatedly to defer its own explanations (“For information about using ScanDisk, type the following at the command prompt: `help scandisk`”).³ This is a minor point compared to the vocabulary questions raised above, but it completes the picture Chapters 2 and 3 built: DOS’s documentation apparatus by the mid-1990s was not merely divergent from Unix’s `man` page tradition in surface naming (`help` versus `man`, itself a difference in the same register as `COPY` versus `cp`, a full word against an abbreviation) but had developed its own internally consistent cross-referencing structure, manual chapters pointing to on-line help topics pointing back to manual chapters (as in the *Configuring Your System* chapter’s own forward references), a closed referential loop with no point of contact with the `man` page corpus a Unix-trained reader would already know how to navigate.

4.4 What a Late Arrival Demonstrates

The chapter’s significance for this book’s overall argument lies less in any individual piece of vocabulary than in the timing. Delete Sentry, in 1993, and ScanDisk’s `rename`, in late 1993, cannot be explained by CP/M inheritance in the way Chapter 1 explained the slash, since neither CP/M nor early DOS had anything resembling either feature; there was no legacy syntax to preserve and no already-shipped software whose command-line parsing these decisions had to avoid breaking. These were greenfield naming decisions, made twelve and thirteen years into DOS’s life, by a documentation and product-naming apparatus that had every opportunity to reach for Unix’s existing, terse, technical vocabulary (`fsck`-style naming for a disk checker; a plain `restored/not-restored` status for a recovered file) and instead reached, more strongly than at any point examined in the previous two chapters, for the register of consumer product marketing: quality grades, spelled-out product names, a help system built as a closed loop referring only to itself. If Chapter 2’s command-name evidence could still be read as habit persisting from 1981, and Chapter 3’s memory-management evidence as a genuine engineering problem that happened to generate a dense vocabulary as a byproduct, this chapter’s evidence is harder to read as anything other than a documentation culture that had, by 1993, fully naturalized its own idiom and had stopped treating Unix’s parallel vocabulary as a relevant point of reference at all. Part IV takes up that naturalization directly as the book’s synthesis, asking what a decade of vocabulary divergence, memory-management complexity, and product-register naming did for Microsoft’s platform position, whether or not any of it was planned as such.

³*Microsoft MS-DOS Concise User's Guide*, p. 39.

Chapter 5

Naming That Travels

The four chapters preceding this one have been built almost entirely from divergence: commands spelled out where Unix abbreviates, switches where Unix has flags, a memory-management vocabulary with no Unix counterpart at all, a recovery apparatus reaching for consumer-product language rather than technical description. A book built from divergence alone risks overstating its case, and risks it in a specific way: a reader could reasonably suspect that any operating system, examined closely enough, would show some vocabulary divergence from any other, and that this book has simply gone looking for it. This chapter takes up the opposite evidence directly, the substantial vocabulary DOS did share with Unix at the level of the file and directory system, because the shared vocabulary turns out to make the book's argument in a more interesting way than further divergence would have.

5.1 Where the Words Actually Match

The *Microsoft MS-DOS Concise User's Guide*'s introductory chapter on file organization uses a set of terms that would be immediately legible to a Unix user of the same period with no translation required. "Strictly speaking, all directories are subdirectories, except for one, which is called the root directory,"¹ the manual explains, introducing *root directory* as the term for the top of the tree, exactly the term Unix documentation of the same period used for exactly the same structural role. The manual's account of a *path* as the sequence of directories leading from the root to a file, illustrated with a tree diagram (C: branching to RECIPES, then to ENTREES, BREADS, and DESSERTS, with LASAGNA.TXT as a leaf) is, at the level of the concept being taught, identical to a Unix path from / through a chain of subdirectories to a file. The manual's treatment of *wildcards*, the asterisk standing for a run of characters and the question mark for a single character, uses the same word, the same two symbols, and describes the same intuitive behavior (match everything, match one character in this position) that Unix shell globbing offers under an identical vocabulary.

This is worth pausing on because it directly narrows Chapters 2 through 4's claim. Those chapters did not argue, and this book has not claimed anywhere, that Microsoft's engineers and documentation writers were incapable of using Unix-legible vocabulary, or unaware of it, or reflexively hostile to it as a matter of course. Chapter 1 already established that

¹*Microsoft MS-DOS Concise User's Guide*, p. 12.

Microsoft adopted the Unix-derived directory tree wholesale in DOS 2.0, `.` and `..` included. This chapter’s file-organization vocabulary, root directory, path, wildcard, extends that same adoption into the words used to teach the structure to users, not merely the structure itself. Where DOS needed a new conceptual vocabulary for something Unix workstations of the period did not need to solve at all, hierarchical storage on a machine whose users had never encountered directories, and where Unix’s own terminology was already the obvious, available, non-proprietary answer, DOS took it.

5.2 The 8.3 Boundary

The convergence does not extend to the mechanism underneath the shared words, and the manual’s own naming-rules section shows exactly where it stops. A DOS filename, per the guide, can be “up to eight characters long,”² with an optional extension of up to three characters, is not case-sensitive, and excludes a specific list of characters (spaces, commas, backslashes, and several others) that Unix filenames tolerate without difficulty. Unix, from its earliest filesystem implementations onward, imposed a length limit far more generous even in the 1970s, treated case as significant, and gave the period no structural role at all, a period in a Unix filename is simply a character like any other, meaningful only by convention (`.tar.gz`, multiple periods, no periods) rather than by filesystem enforcement.

The consequence is that *extension* does different work in the two systems even where both traditions use the word. A classic FAT directory entry stores eleven name bytes in an eight-byte base-name field and a three-byte extension field; the dot displayed to users is not itself stored between them. The extension is therefore structurally represented, but it should not be confused with metadata such as the attribute byte. DOS software then gives particular suffixes operational meaning: `COMMAND.COM`, for example, recognizes `.COM`, `.EXE`, and `.BAT` when resolving executable commands. Unix filesystems generally treat periods and suffixes as ordinary filename characters, while executable formats and permissions are determined independently. The distinction is between a fixed on-disk naming field reinforced by command behavior and a convention interpreted by applications, not between “metadata” and “mere naming.”

5.3 Wildcards That Look Alike and Compute Differently

The wildcard case makes the same point more sharply, because the divergence here is not merely conceptual but produces genuinely surprising behavior that DOS users of the period had to learn to work around. Unix shell globbing is handled by the shell itself, before the target program ever runs: the shell expands `*.txt` into an explicit list of matching filenames and hands that list to the program as ordinary arguments, a process entirely separate from and prior to whatever matching logic, if any, the program itself implements. DOS’s wildcard matching, by contrast, descends directly from CP/M’s File Control Block interface, in which a filename pattern was converted into an eleven-character positional template, eight characters for the name and three for the extension, with each position resolved independently as either a literal character to match exactly or a question mark

²*Microsoft MS-DOS Concise User’s Guide*, p. 14.

standing for any single character; the asterisk was not a primitive of this matching scheme at all but a documentation-level shorthand that got expanded into a run of question marks filling out the rest of its field before matching ever began.

This produced behavior with no equivalent in ordinary Unix shell expansion. The clearest example is `*.*`: in classic DOS usage it became the conventional request for all ordinary filenames, including names with no displayed extension, because the two fixed fields were wildcarded. In a Unix shell, `*.*` ordinarily selects only visible names containing a literal period, while `*` is the broader pattern. DOS programs could also implement their own parsing, so behavior was not perfectly uniform across every utility. The important difference is architectural: Unix shells normally expand globs before launching a program, whereas DOS commonly passed patterns to a command or system service whose matching rules reflected the 8.3/FCB lineage.

5.4 What Shared Vocabulary Is Evidence Of

The chapter's argument, then, is not that this book's central claim was wrong to find divergence pervasive, but that the shape of the convergence it does find sharpens rather than undercuts that claim. Where DOS adopted Unix's words for a genuinely new problem, hierarchical organization on a machine with no prior directory concept at all, it adopted them cleanly, root directory and path meaning in DOS exactly what they meant in Unix documentation, because there was no CP/M legacy, no already-shipped software, and no accumulated FCB-era interface standing in the way of a clean borrowing. Where DOS instead inherited an existing mechanism from CP/M, the eleven-character FCB template underlying both filenames and wildcard matching, the vocabulary that traveled (wildcard, asterisk, question mark) traveled without the mechanism traveling with it, leaving two systems that taught users the same words for what turned out to be, at the moment those words met an edge case, two different computational objects. This is the inverse of Chapters 2 through 4's pattern rather than a counterexample to it: there, identical concepts received different names; here, identical names concealed different mechanisms, and in both directions the same underlying fact governed the outcome, whether CP/M had already committed the interface to a specific shape before the question of Unix convergence ever arose. Part IV draws this and the preceding chapters' evidence together into the book's synthesis.

Part I

Synthesis

Chapter 6

Vocabulary as Moat

The chapters preceding this one have circled a single question from five different directions without ever resolving it directly: not whether DOS's vocabulary diverged from Unix's, which the evidence has established repeatedly and in detail, but what kind of fact that divergence is. This closing chapter takes up that question as its sole business, and does so by refusing the two easiest answers available. The first easy answer is that none of it was intentional, an accumulation of independent accidents each explicable on its own narrow terms, CP/M inheritance, hardware constraint, documentation habit, with no larger pattern binding them together beyond this book's own retrospective narration. The second easy answer is that all of it was intentional, a coordinated strategy pursued by a small number of decision-makers at Microsoft who understood from the outset that proprietary vocabulary would function as a switching-cost weapon and built DOS's documentation accordingly. Both answers are too easy because both let the book stop looking at the actual mechanism by which vocabulary divergence, whatever its origin, came to matter.

6.1 Restating the Evidence Without the Narrative

Before proposing a mechanism, it is worth restating what the preceding chapters actually showed. Chapter 1 traced DOS's early shape to CP/M compatibility while showing, from Microsoft's own DOS 2.0 materials, that Unix-like forward-slash paths existed during development before the IBM-facing backslash convention prevailed. Chapter 2 showed that inherited command names became an interface competitors had to reproduce; it did not establish that Microsoft selected names to avoid Unix abbreviations. Chapter 3 showed that memory-management burdens arose from real hardware and compatibility constraints while leaving room for competing vendors to redistribute some of that burden through better automation. Chapter 4 identified a late consumer-facing product register. Chapter 5 showed that shared words such as *path* and *wildcard* could travel even when their underlying implementations differed. Together these cases support an argument about installed knowledge and interface stability, not a claim that lexical divergence itself proves strategic intent.

Read together rather than chapter by chapter, this evidence supports a narrower and more defensible claim than either of the two easy answers above. It is not that Microsoft

coordinated a vocabulary strategy from 1981 onward; Chapter 1's account of the backlash and Chapter 5's account of the directory tree both show real, uncoordinated, path-dependent decision-making producing outcomes nobody could have planned in advance. It is also not that the resulting divergence was inert, a historical curiosity with no bearing on anything beyond itself; Chapter 3's market evidence from Quarterdeck and Digital Research shows that the complexity DOS's own vocabulary sustained had real competitors willing to sell simplicity instead, which means DOS's complexity was a choice with alternatives, whatever the process that produced the choice.

6.2 The Mechanism: Accumulated Switching Cost Without a Switching Decision

The mechanism this book proposes, tentatively and as a closing synthesis rather than a claim requiring further chapters to establish, is that vocabulary divergence and switching cost can enter a reinforcing relationship without any party along the way making a decision that, examined on its own, looks like a strategic choice to build the relationship. Consider the position of a documentation writer revising the MS-DOS 5.0 manual for MS-DOS 6.0 in 1993, deciding whether to keep calling a command-line option a switch or to start calling it a flag. Chapter 2 established that this specific change would have broken no software; the cost of making it was almost entirely a cost paid by the writer and by whatever internal review process existed, the cost of noticing the divergence, deciding it was worth fixing, and defending that decision against the null hypothesis that the existing word had served perfectly well for a decade. The cost of not making it was paid by someone else entirely: a user trying, at some later point, to move skills from DOS to Unix, or a corporate IT department trying to train staff across both systems, or, more diffusely, the general software ecosystem's ability to treat expertise in one system as partially transferable to expertise in the other. No individual instance of this asymmetry, one writer, one word, one manual revision, constitutes a strategy. The asymmetry recurring identically across hundreds of such individual choices, made independently by different people across a decade, for the same structural reason each time, produces an outcome indistinguishable in its effects from a strategy, without requiring anyone to have held the strategy in mind.

This is the sense in which the title should be read. A moat need not have been dug for every later use to which it is put; an inherited river can protect a fort. By the early 1990s DOS vocabulary formed part of a larger compatibility surface comprising commands, batch files, manuals, applications, drivers, training, and support practice. Vocabulary alone was a shallow barrier, but it indexed investments that were not shallow. The function performed and the intent behind it are separable questions: the evidence supports accumulated interface dependence and competitive effects more strongly than it supports deliberate lexical separation.

6.3 What the Moat Bought, and for Whom

It is worth being concrete about what accumulated switching cost is worth, because the abstraction can obscure a very ordinary commercial fact. A corporate customer in 1993 who

had invested years of staff training in DOS's `switch`, `DEVICE=`, and `HIMEM.SYS` vocabulary, whose internal documentation, internal tooling, and internal hiring practices had all been built around DOS-specific fluency, faced a real cost in migrating any part of that investment to a Unix-based alternative, a cost with no necessary relationship to the technical merits of either system and entirely a function of how much DOS-specific vocabulary that customer's own institutional knowledge had accumulated. This cost was not visible on any Microsoft invoice, and it did not need to be, because it was paid in advance, gradually, by the customer's own choice to keep using DOS, each individual choice reasonable in isolation and cumulatively producing a switching cost that Microsoft did not have to build, price, or even name, because the customer's own accumulated fluency built it unassisted.

This is, this book argues, the real payoff of vocabulary divergence sustained past its originating cause, and it explains why the individually low-cost decisions traced above kept resolving in the same direction for a decade without requiring central coordination. Reconciling DOS's vocabulary with Unix's would have reduced the accumulated switching cost DOS's own installed base represented, a cost that functioned, whether intentionally cultivated or not, entirely in Microsoft's favor and entirely at the expense of a customer base whose own training investment was the actual source of the lock-in. No single decision-maker needed to see this whole picture for the pattern to persist; each writer, each engineer, each product manager needed only to notice that reconciliation carried a cost falling on them personally (the labor, the internal debate, the risk of confusing an existing user base) and no comparably salient benefit falling on them personally, since the benefit of reduced switching cost accrued to users and to competitors, not to Microsoft's own documentation team.

6.4 The Limits of the Argument

This book has tried, at several points, most explicitly at the end of Chapter 2, to state plainly what its evidence does not establish, and this closing chapter should do the same rather than let the momentum of a concluding argument overstate the case. This book has not identified a founding document, a strategy memo, or a named decision-maker who articulated the vocabulary-as-moat mechanism this chapter proposes and then executed it. It has not shown that Microsoft's competitors in the same period, IBM's own PC-DOS branch, Digital Research, or others, behaved differently in every respect; Chapter 2's DR-DOS comparison showed only that Digital Research chose not to converge toward Unix either, which supports the claim that Unix-divergence was not commercially compelled but does not show DR-DOS was free of comparable dynamics in its own right. And it has not attempted the harder comparative project of showing that DOS's vocabulary divergence was larger, in some measurable sense, than the vocabulary divergence any two independently developed operating systems of the period would be expected to show as a simple consequence of independent development, a baseline this book has gestured toward (Chapter 5's convergence evidence, showing where DOS did borrow cleanly, serves partly this function) without establishing rigorously.

What this book has established, and what it takes its five preceding chapters to have earned, is narrower and, this closing chapter argues, sufficient to justify the book's title without overreaching past it: that DOS's vocabulary divergence from Unix was neither uniformly forced by technical necessity nor uniformly absent of any pattern; that the

pattern, where it existed, tracked cases where reconciliation was individually cheap and its benefits fell on someone other than the party deciding whether to reconcile; and that the resulting vocabulary, sustained past the expiration of its originating causes, functioned as an accumulated switching cost with real market consequences for anyone who had invested in learning it, whether or not any person at Microsoft ever named that function or intended it. A moat need not be dug on purpose to hold water.

Part II

Coda

Chapter 7

The Longer Arc: From the Hobbyists to the Kernel

The preceding six chapters have kept their focus narrow by design, confined to MS-DOS’s own vocabulary and its own decade and a half of documented history. That narrowness was a methodological choice, not a claim that DOS’s story stands alone. This closing chapter widens the frame deliberately, tracing the same enclosure-and-release logic Chapter 6 proposed as DOS’s mechanism across a much longer span of Microsoft’s institutional history, from a 1976 letter written before DOS existed to a 2018 GitHub repository that finally gave DOS’s own source code away. The claim this chapter makes is not that DOS’s vocabulary divergence caused or was caused by any of these later episodes; it is that the same underlying asymmetry, between the party who benefits from enclosing a shared resource and the party who bears the cost of that enclosure, recurs at every scale this book has been able to check it against, from a single command name to an entire operating system category.

7.1 1976: The Letter Before the Company Had a Product to Enclose

Microsoft’s founding relationship to shared computing resources was established before MS-DOS existed and before the company had shipped an operating system of any kind. In February 1976, Bill Gates, then a general partner in the small Albuquerque firm still spelled Micro-Soft, circulated “An Open Letter to Hobbyists” to the Homebrew Computer Club and several hobbyist publications, complaining that hobbyists were copying the company’s Altair BASIC interpreter freely rather than paying the licensing fee, estimating that unpaid copies outnumbered paid ones roughly ten to one.¹ The letter’s central claim, that hardware must be paid for but software was widely treated by the hobbyist community as something to share freely, named directly the norm Gates was writing to overturn: a commons of freely circulating code among the Altair-owning hobbyist clubs of 1975, a commons that

¹Gates, “An Open Letter to Hobbyists,” *Homebrew Computer Club Newsletter* 2, no. 1 (February 3, 1976); also published in *MITS Computer Notes* (February 1976), *People’s Computer Company* (March 1976), and *Radio-Electronics* (May 1976).

predated any commercial software industry and that Gates’s letter proposed to replace with a licensing regime.

This book does not take a position on whether Gates’s argument, that unpaid copying would starve future software development of the revenue needed to sustain it, was correct as economics; it was, on its own terms, a defensible argument in 1976, when no commercial software industry existed to demonstrate an alternative. What the letter establishes for this book’s purposes is a founding institutional stance, adopted before DOS, before Xenix, before any of the vocabulary this book has spent six chapters documenting: that a resource’s status as freely shared was, for this company, a starting condition to be corrected rather than a default to be preserved, whenever the company held rights capable of correcting it.

7.2 1980–1987: A Real Unix, Held and Released

The clearest comparison at the scale of an entire operating system is Microsoft’s ownership of Xenix. Microsoft licensed Unix source from AT&T in 1979 and announced Xenix in 1980; through ports and partners including SCO, Altos, and Tandy, it became a widely deployed microcomputer Unix.² This represented years of genuine engineering investment in a Unix-derived operating system at a company simultaneously developing DOS. Xenix itself, however, was commercially licensed proprietary software. Unix ancestry must not be confused with open-source licensing, a category that did not yet organize the market in its later sense.

As Microsoft and IBM committed to OS/2, Xenix’s internal priority fell; in 1987 Microsoft transferred responsibility and rights to SCO while retaining an equity interest. That transaction is consistent with a change in platform priorities, but it is not proof of a general rule about releasing shared resources. Xenix was sold to another vendor, not released into a commons. The episode belongs here as a contrast: Microsoft possessed deep Unix expertise while choosing to keep the DOS and Xenix product families distinct, so DOS’s nonconvergence cannot be attributed to ignorance of Unix.

7.3 The 1990s: Embrace, Extend, and Extinguish

The pattern recurs a decade later at the scale of an entire communications medium. As the World Wide Web emerged as an open, standards-governed commons in the early-to-mid 1990s, built on protocols (TCP/IP, HTTP, HTML) that no single company controlled, Microsoft’s initial position was, by its own later account, one of underestimation, followed by an aggressive campaign once the web’s significance became clear. The company’s internal strategy for this campaign became public through discovery in the *United States v. Microsoft Corp.* antitrust proceedings of the late 1990s, in which testimony from an Intel vice president, Steve McGeady, described Microsoft executives using the word “extinguish” to describe the intended endpoint of a strategy an internal document phrased more blandly as “embrace, extend, and innovate.”³ Internet Explorer, bundled with Windows and extended with

²On Xenix’s contemporary reach, see Samuel J. Leffler et al., *The Design and Implementation of the 4.3BSD UNIX Operating System* (Addison-Wesley, 1989).

³The distinction between the internal document’s phrasing (“embrace, extend, and innovate”) and McGeady’s testimony that Microsoft executives used “extinguish” verbally in a 1995 meeting with Intel is

proprietary capabilities such as ActiveX and non-standard document object model behavior, was the primary vehicle: web developers who wanted their pages to render correctly for the majority of users increasingly had to target Internet Explorer’s specific, non-standard behavior rather than the open specifications a standards body had published, a dynamic that contributed materially to Netscape Navigator’s commercial decline over the following several years.

This episode differs from Xenix in an important respect. Microsoft never owned HTML or the web protocol stack, so the relevant conduct was extension and distribution rather than divestment. Internet Explorer combined standards support with proprietary facilities and the reach of Windows. The antitrust record supports claims about maintaining the Windows applications barrier to entry; it does not turn every browser incompatibility into proof of one slogan or make the episode a direct continuation of DOS command naming. It is best treated as a later, documented case in which control of an interface and control of distribution reinforced one another.⁴

7.4 2014–2018: What Got Released, and When

The change in posture beginning in the mid-2010s under Satya Nadella is real and should not be dismissed as mere public relations. Windows Subsystem for Linux, announced in 2016, initially translated Linux system calls for unmodified user-space binaries. WSL 2, announced in 2019 and released in 2020, instead runs a Microsoft-maintained build of the Linux kernel inside a lightweight virtual machine.⁵ This is materially different from reimplementing selected Unix-like facilities under a wholly separate interface. It is not literally an unmodified upstream kernel, but it is real Linux source shipped and maintained as part of Windows.

The same period saw Microsoft acquire GitHub outright, in a deal announced in June 2018 and closed that October, and, in the same September that the acquisition closed, re-publish MS-DOS’s own source code, versions 1.25 and 2.0, on GitHub under the MIT license, expanding on a more restricted, non-commercial release the company had made through the Computer History Museum back in March 2014.⁶ The code this book’s Chapters 1 through 5 has spent its evidence reconstructing from secondary historical accounts and the era’s user manuals is, as of that release, available in its original assembly, line by line, to anyone who wants it, with no restriction beyond a request, honored more as courtesy than enforcement, not to submit pull requests to a codebase Microsoft has explicitly said it will not maintain.

documented in the *United States v. Microsoft Corp.* trial record and summarized in, among other sources, the Wikipedia entry “Embrace, extend, and extinguish.”

⁴See *United States v. Microsoft Corp.*, 84 F. Supp. 2d 9 (D.D.C. 1999), Findings of Fact, especially the discussion of the applications barrier to entry and Microsoft’s response to middleware threats.

⁵Microsoft’s `WSL2-Linux-Kernel` repository contains the kernel source and configuration used for WSL 2, derived from upstream Linux and distributed under the applicable free-software licenses.

⁶Computer History Museum, “Microsoft MS-DOS Early Source Code,” noting the original March 2014 release and the September 2018 GitHub re-release; Microsoft’s own Windows Command Line blog post, “Re-Open-Sourcing MS-DOS 1.25 and 2.0,” confirms the same timeline and the shift to the MIT license.

7.5 The Lifecycle the Longer Arc Reveals

Read end to end, these episodes do not establish a single invariant corporate law. They do show that Microsoft's posture toward interoperability changed with the location of its commercial advantage. Xenix was divested rather than opened; Internet Explorer coupled standards participation to Windows distribution; WSL incorporated Linux when Linux compatibility increased Windows' value; and early DOS source was released only after its competitive importance had expired. The recurring object is not "openness" in the abstract but control over the interface at which users, developers, and applications meet a platform.

The longer arc therefore supplies comparison rather than proof by recurrence. Each decision—licensing BASIC, divesting Xenix, extending browser capabilities, incorporating Linux compatibility, and publishing obsolete DOS source—answered to different technical and commercial conditions. Their common feature is that compatibility has economic value and that firms prefer to govern interfaces central to their advantage. DOS vocabulary is one small and unusually visible layer of that broader process. Treating it at that scale preserves the book's political insight without requiring four decades of heterogeneous decisions to be moves in one unbroken game.

Chapter 8

The Interior and the Seal

The chapters above have treated MS-DOS as a command-line system whose vocabulary users typed directly, and the longer arc traced in Chapter 7 as a story told mostly at the level of corporate strategy and public record. This closing chapter fills a gap between those two registers: what DOS's vocabulary and configuration apparatus meant for the enormous population of users who never typed a DOS command directly at all, because by the time they touched a computer, Windows had already put a graphical shell in front of it, and what the retail experience of buying software for that computer, or simply seeing it advertised, communicated to people, including people with no computer at hand to run it on.

8.1 Windows as a Program DOS Ran

Windows 3.0 and 3.1, the versions that brought graphical computing to a mass consumer market between 1990 and 1992, were not, architecturally, a replacement for DOS. They were a program DOS ran. A user booted MS-DOS first, exactly as Chapters 1 through 5 of this book have described it, and then typed `WIN` at the DOS prompt to launch the graphical environment, which loaded as a large, sophisticated DOS application occupying the memory Chapter 3 spent an entire chapter documenting the struggle to free. The Program Manager and File Manager, the graphical replacements for typing `DIR` and navigating with `CD`, displayed and manipulated the exact same underlying file system Chapter 5 examined: the same drive letters, the same backslash-separated paths, the same eight-character-name-plus-three-character-extension limit, now represented as folder icons and file icons rather than as a printed directory listing, but governed by identical rules. A user who never once opened a DOS prompt in Windows 3.1 was nonetheless bound, at every turn, by DOS's naming vocabulary; the icon simply supplied a picture where the command line had supplied a word, without changing what the word beneath the picture was allowed to say.

Windows 95 complicated this relationship without severing it. Marketed and widely understood as a genuine break from DOS, a real 32-bit operating system in its own right, Windows 95's actual architecture was a hybrid that retained MS-DOS as a functioning layer underneath its new kernel rather than replacing it outright. The system's core, `VMM32.VXD`, ran in 32-bit protected mode and handled memory management, interrupts, and the loading of the new 32-bit virtual device drivers that gave Windows 95 its improved stability and

performance, but the boot process that got a machine to the point of loading `VMM32.VXD` still ran a genuine, if quietly renumbered, version of MS-DOS first: Windows 95's own `IO.SYS` identified itself as MS-DOS 7.0, and any device or piece of legacy software without a native 32-bit driver still fell back to real-mode DOS to function.¹ Caldera, the company that by the mid-1990s owned the competing DR-DOS lineage discussed in Chapter 2, argued in litigation that Windows 95 was, on this basis, not a single integrated product at all but MS-DOS 7.0 and a separate Windows 4.0 packaged to look like one, a characterization Microsoft disputed but which the underlying architecture, DOS booting first and remaining resident throughout the session, does not straightforwardly refute.²

Windows 95 does mark a genuine break from the vocabulary Chapter 5 examined, and it is worth crediting precisely: the system introduced VFAT, a filesystem extension supporting long file names well beyond the eight-plus-three limit, the first structural change to DOS's naming rules since the extension field itself was introduced in 1981. But VFAT did not replace the old 8.3 format; it layered long names on top of it, generating a hidden, backward-compatible short name (the familiar `PROGRA~1` pattern) alongside every long one, so that older DOS-era software, and the FCB matching scheme Chapter 5 traced to CP/M, would continue to find something to match against. The interior this section has been describing, DOS booting first, DOS's naming rules still enforced somewhere beneath every long file name, did not disappear with Windows 95's graphical polish. It was, more precisely, finally allowed to grow a second, more generous name alongside the name it had always required.

8.2 The Seal on the Box

The vocabulary and configuration apparatus this book has spent seven chapters tracing had a second life outside the machine entirely, on the packaging of software sold at retail, in a period when a very large number of people encountered that packaging, in a store, long before they owned a computer capable of running what it advertised. Microsoft formalized this retail signal with the “Designed for Windows 95” logo program, launched alongside the operating system itself in 1995: software that passed an independent certification process, which required, among other things, that a program avoid 16-bit code where a 32-bit alternative existed, support the new long file names, and use the Windows registry rather than DOS-era `.INI` configuration files, earned the right to display a specific flag-shaped logo prominently across its box.³ Unlike the more diffuse “MS-DOS Compatible” branding of the preceding decade, this was an enforced technical standard with a visible seal attached to passing it, and the seal's real audience was not the buyer's technical knowledge but the buyer's trust: a shopper with no way of independently verifying whether a given game would run correctly, or a shopper with no computer at all yet, browsing a shelf in anticipation of

¹Windows 95's boot sequence and the MS-DOS 7.0 identification in `IO.SYS` are documented in the Wikipedia entry “MS-DOS 7” and corroborated by contemporary architecture references, e.g., the Windows 95 architecture notes archived at users.skynet.be.

²Caldera's characterization of Windows 95 as two bundled products is summarized in the “MS-DOS 7” Wikipedia entry, citing Caldera's own public statements during its antitrust litigation against Microsoft over DR-DOS.

³The Designed for Windows 95 program's requirements and its prominence on packaging are described in Steven Sinofsky's *Hardcore Software* newsletter, entry “038. Designed for Windows 95,” and in Raymond Chen's *The Old New Thing* devblog post comparing the program's certification holograms.

the machine a birthday or a holiday might bring, could read the logo as a promise that the box's contents had cleared a bar someone else had already checked.

The certification program connects to the earlier vocabulary argument more concretely than the logo alone suggests. A developer seeking the mark had to satisfy Microsoft-defined requirements concerning Win32 behavior, long filenames, installation, and system integration. Some underlying technologies implemented published specifications or broadly available APIs; the certification program itself was nevertheless a platform-owner program tied to Microsoft's current engineering priorities. Its effect was double-edged: it reduced uncertainty for buyers and encouraged more consistent software, while also allowing the platform owner to turn its preferred practices into a visible condition of market trust.

8.3 The Weight of a Single Environment Variable

Nothing in this book's earlier chapters made vivid what the abstract vocabulary of Chapter 3's memory-management apparatus actually cost an ordinary user attempting to get a piece of software running, and the sound card offers the clearest single example. Installing a Sound Blaster or Sound Blaster-compatible card through the first half of the 1990s required setting a DOS environment variable, conventionally named **BLASTER**, in **AUTOEXEC.BAT**, in a format that packed five or more separate hardware parameters into a single unglossed string: a base port address, an interrupt request line, a low DMA channel, sometimes a second, high DMA channel for 16-bit audio, a MIDI port address, and a numeric code identifying the specific card model, commonly rendered as something like **SET BLASTER=A220 I5 D1 T6**.⁴ Each letter-prefixed field corresponded to a physical resource, an interrupt line, a direct memory access channel, that had to be free of conflict with every other device already installed in the machine, a fact a user typically discovered only when a game launched with no sound, or with distorted sound, or locked up entirely, and had to be diagnosed and corrected by hand, often with a jumper setting on the physical card itself in addition to the software-side variable.

This single line of **AUTOEXEC.BAT**, unglossed, unforgiving of a wrong digit, and entirely of a piece with the **DEVICE=**, **DOS=HIGH**, **UMB**, and switch-heavy vocabulary Chapter 3 examined in the abstract, was for a great many users the actual, felt texture of DOS's configuration apparatus: not a chapter in a manual, but the difference between a game working and not working on the specific afternoon it mattered. The logo on the box promised that the software inside had been checked against a standard; nothing on that box explained the **BLASTER** variable a user might still need to get right before the promise could be redeemed. The distance between the two, a seal of institutional trust on the outside and an unglossed hardware-configuration string on the inside, is, in miniature, the same distance this book's earlier chapters have located throughout DOS's vocabulary at every scale examined: the confidence a system's surface offered, and the specialized fluency its interior actually required.

⁴This example follows the widely-documented convention described in, among other sources, the *DOS Wiki*'s "BLASTER Variable" entry and the Lo-tech Wiki's "BLASTER Environment Variable" page.

Chapter 9

Zoom, Tutorial, Homophone

This closing chapter gathers three smaller observations that do not extend the book’s central argument so much as fill in texture the preceding eight chapters left out: what it felt like to discover that a picture, unlike a drawing on paper, could be entered and edited at the level of its individual constituent marks; how an entire generation’s hands were taught, deliberately and by name, the specific gestures this book has spent so many pages examining the vocabulary of; and a small linguistic accident in the name “DOS” itself that neither Unix nor CP/M had any part in creating.

9.1 Paintbrush and the Pixel Beneath the Line

Microsoft Paintbrush, bundled with Windows since version 3.0, offered a zoom function fixed at eight-times magnification, switching the working canvas from a smooth drawn line to a visible grid of individual colored squares that could be edited one at a time before zooming back out to see the image whole again.¹ This was not a large or sophisticated feature by the standards of later image editors, and Paintbrush itself was, by most contemporary and retrospective accounts, a fairly limited program, unable to interpolate fast mouse movement into a smooth curve, prone to jagged lines, closer to a toy than a professional tool. But the specific capability the zoom function offered, entering a picture and discovering that it was, underneath, made of discrete, individually addressable units that could each be changed without disturbing the picture’s overall shape, had no equivalent in drawing on paper. A pencil line, examined closely enough, is graphite worn into paper fiber, not a grid of separable elements a hand can select and recolor one at a time; the closest paper-based analogy, cross-hatching or stippling under a magnifying glass, still offers nothing to select and replace, only more of the same continuous mark to look at more closely. Paintbrush’s zoom function demonstrated something conceptually new: that an image could be a discrete structure, addressable at the level of its smallest components, in a way no prior drawing medium had made available to a child holding a mouse for the first time.

¹Windows 3.x Paintbrush’s zoom is documented as fixed at 8x magnification in TheAlmightyGuru’s retrospective review of the program, and its zoom-in behavior (scaled to the size of the current image) is described in Microsoft’s own Knowledge Base article Q66135.

9.2 The Tutorial Taught the Hand a New Grammar

Windows 3.1 shipped with a built-in interactive tutorial, accessible from Program Manager’s Help menu, structured in two parts that this book’s earlier chapters’ vocabulary can now be read against directly. The first part, a “mouse lesson,” asked the user to physically practice each individual gesture the interface would require before any actual task was attempted: moving the mouse, left-clicking, double-clicking, and dragging, each demonstrated against a simple on-screen target (the tutorial’s own control-menu box) before the lesson would advance.² The second part, a “Windows basics lesson,” moved from raw gesture to named interface operations, walking the user through opening a program group, starting the bundled Write application, and then practicing, as discrete, checked steps, maximizing a window, restoring it, minimizing it, moving it, resizing it, and opening a menu to select a command, each step confirmed complete before the tutorial proceeded to the next.

This tutorial deserves to be read as more than a usability aid. It was, in effect, a formal curriculum teaching an entire generation of new computer users the specific embodied vocabulary, click, double-click, drag, minimize, maximize, restore, that Chapter 2’s command-line vocabulary had its own list of typed equivalents for a decade earlier. Where Chapter 2 traced `COPY`, `DEL`, and `DIR` as words a user had to already know to type, the Windows 3.1 tutorial made the graphical vocabulary’s equivalent gestures something the operating system itself taught explicitly, step by step, confirmed correct before moving on, to a population many of whom had never touched a mouse at all before sitting down at that specific lesson. The interesting comparison is not to DOS but sideways, to the Super NES Mouse, released in 1992 bundled with Nintendo’s *Mario Paint*, one of the first uses of a mouse peripheral on a game console rather than a personal computer, and packaged, like Paintbrush, around a drawing and animation tool rather than around productivity software.³ A child who had never sat at a Windows machine and worked through its mouse lesson could nonetheless learn the identical physical gesture, point, click, drag, at a television screen, through a console cartridge whose central activity was also drawing. The vocabulary of the mouse, in other words, was being taught simultaneously and independently on two entirely different platforms in the same historical moment, converging on the same small set of physical gestures because the device itself, not any operating system’s naming choices, was what actually constrained what a hand could be taught to do with it.

9.3 A Note on the Name

One naming collision this book’s earlier chapters had no occasion to examine, because it involves no comparison to Unix at all, is worth recording before this monograph closes. “DOS,” pronounced in English as a single syllable rhyming with “boss” or “loss,” is also, in Spanish, an ordinary and extremely common word: *dos*, the number two, pronounced with a pure, undiphthongized vowel closer to the vowel in English “dose” than to the vowel English

²The structure of Windows 3.1’s built-in mouse lesson and Windows Basics lesson, including the specific sequence of practiced actions, is preserved in the BetaWiki entries for Windows 3.1 builds 068 and 103.

³The Super NES Mouse’s 1992 release, bundled with *Mario Paint*, and its status as an early instance of mouse-based control on a console are documented in the *Super NES Mouse* Wikipedia entry and the *Mario Paint* entry on Super Mario Wiki.

speakers actually use when they say the acronym “DOS” aloud. The English acronym and the Spanish numeral are spelled identically and share no etymological connection whatsoever, Microsoft’s name being an initialism for “disk operating system” with roots in Gary Kildall’s and Tim Paterson’s naming choices for CP/M and 86-DOS respectively, entirely disconnected from any Romance-language numeral. But the visual identity of the two words, four letters, no diacritics, identical in every written context, meant that for the hundreds of millions of Spanish speakers who encountered Microsoft’s operating system throughout the 1980s and 1990s, the product’s name was, on the page if not in the mouth, indistinguishable from the second-most basic counting word in their own language, a coincidence with no analogue in this book’s earlier vocabulary chapters and no bearing on the enclosure argument those chapters built, but a reminder that the vocabulary a company settles on for its own internal reasons can acquire meanings, in languages the company was not designing for, that no amount of CP/M inheritance or Unix-avoidance can explain.

Bibliography

Caldera / DR-DOS litigation. Caldera’s characterization of Windows 95 as MS-DOS 7.0 and a separate Windows 4.0 bundled to appear as one product, made during its antitrust litigation against Microsoft, is summarized in the Wikipedia entry “MS-DOS 7.”

Chen, Raymond. *The Old New Thing* (devblog). “Comparing the two holograms on the Windows 95 box,” on the “Designed for Windows 95” certification program.

Computer History Museum. “Microsoft MS-DOS Early Source Code.” Notes the original March 2014 non-commercial release of MS-DOS 1.25 and 2.0 source, and the September 2018 GitHub re-release.

DOS Wiki (Fandom). “BLASTER Variable.” Documents the Sound Blaster environment-variable convention (base port, IRQ, DMA channel, card type).

Gates, Bill. “An Open Letter to Hobbyists.” *Homebrew Computer Club Newsletter* 2, no. 1 (February 3, 1976). Also published in *MIT Computer Notes* (February 1976), *People’s Computer Company* (March 1976), and *Radio-Electronics* (May 1976).

Leffler, Samuel J., et al. *The Design and Implementation of the 4.3BSD UNIX Operating System*. Addison-Wesley, 1989. Cited for Xenix’s contemporary reach as a widely-deployed Unix.

Lo-tech Wiki. “BLASTER Environment Variable.” Field-by-field documentation of the SET BLASTER convention.

Microsoft Corporation. *MS-DOS 2.0 System Calls Reference*, v2.0/bin/SYSCALL.DOC, and v2.0/source/UTILITY.txt. In the published microsoft/MS-DOS source repository, GitHub.

Microsoft Corporation. *Microsoft MS-DOS Concise User’s Guide*. User manual distributed with mid-1990s consumer MS-DOS releases; cited throughout for command documentation, file-organization vocabulary, and configuration instructions (pp. 12, 14, 39, 41, 80).

Microsoft Corporation. Knowledge Base article Q66135, on Paintbrush’s zoom-in behavior in Windows 3.0/3.1.

Microsoft Corporation. WSL2-Linux-Kernel repository, GitHub. Kernel source and configuration used for Windows Subsystem for Linux 2, derived from upstream Linux.

Microsoft Windows Command Line (devblog). “Re-Open-Sourcing MS-DOS 1.25 and 2.0.” Confirms the September 2018 GitHub release timeline and the shift to the MIT license.

Paterson, Tim. “An Inside Look at MS-DOS.” Retrospective account of 86-DOS’s CP/M-facing design.

Sinofsky, Steven. *Hardcore Software* (newsletter). Entry 038, “Designed for Windows 95,” on the certification program’s requirements and retail prominence.

Super Mario Wiki. “Mario Paint.” Documents the 1992 release and its bundling with the Super NES Mouse.

TheAlmightyGuru. “Paintbrush (Microsoft).” Retrospective review documenting Windows 3.x Paintbrush’s fixed 8x zoom and other feature limitations.

United States v. Microsoft Corp. 84 F. Supp. 2d 9 (D.D.C. 1999). Findings of Fact, including the discussion of the applications barrier to entry and Microsoft’s response to middleware threats; trial record also documents Steve McGeady’s testimony distinguishing internal phrasing (“embrace, extend, and innovate”) from verbal use of “extinguish.”

Wikipedia. “Embrace, extend, and extinguish.” Summarizes the DOJ finding and the internal-document/testimony distinction.

Wikipedia. “MS-DOS 7.” Documents the Windows 95 boot sequence and IO.SYS’s MS-DOS 7.0 identification.

Wikipedia. “Super NES Mouse.” Documents the peripheral’s 1992 release bundled with *Mario Paint*.

Windows 95 architecture (skynet.be). Archived architecture notes corroborating the VMM32.VXD/IO.SYS boot architecture.

BetaWiki. Windows 3.1 build 068 and build 103 entries, preserving the structure of the built-in “mouse lesson” and “Windows basics lesson” tutorial.